Which Correction? Resolution Bias and Cross-Generator Detection of AI-Generated Images

Noa Keter · Ido Josephsberg
Deep Learning, Tel Aviv University, 2026b · Dr. Shay Maymon
Code and run instructions: `https://github.com/noa-keter/deep-learning`

1 Motivation and problem definition

A convolutional detector separates real photographs from generated ones almost perfectly when the test images come from the generator it was trained on, and degrades sharply on generators it has never seen. We ask how much of that in-distribution success reflects the *generative process* and how much reflects *benchmark construction*. The concern is concrete: each generator emits images at one fixed resolution while real photographs vary, so the dimensions alone identify both the class and the generator, and a detector can score well without ever examining pixel content. On our data a rule with **zero parameters** — predict "synthetic" if the image is square — reaches **0.977 accuracy on every source–target pair**, which is higher than the cross-generator accuracy of three of the four trained detectors we report below.

Removing that cue is not a neutral act. The generative fingerprint is a high-frequency property, and every available correction damages it in a different way: cropping discards context, rescaling resamples and attenuates the very frequencies that carry the signal, and padding removes the size difference by introducing a border artifact of its own. Prior work applies *one* correction and reports the corrected number. Our contribution is to make the correction itself the experimental variable: we hold the architecture, the data and the protocol fixed, vary only how image size is equalized, and ask whether the conclusions a reader would draw survive that choice. They do not.

2 Related work

Wang et al. [1] established the pattern this project examines: a classifier trained on a single generator detects generated images easily in-distribution, yet its transfer to unseen architectures is uneven. GenImage [2] scaled the question to a million images and formalized the *cross-generator* protocol — train on one generator, evaluate on all others — which is the protocol we adopt. Grommelt et al. [3] then showed that GenImage carries two construction artifacts, JPEG compression and image size, and that removing both raises cross-generator accuracy by more than eleven percentage points for ResNet50 and Swin-T detectors.

We take [3] as established and start where it stops. That work demonstrates that the bias *exists* and corrects it once, by one mechanism. It does not examine whether the choice of mechanism changes the answer. That is our question, and it is a different claim: not that benchmarks are biased, but that **the correction is itself a free parameter that reorders published results**.

3 Data

We use **Tiny-GenImage** (Hugging Face `TheKernel01/Tiny-GenImage`, CC BY-NC-SA 4.0), a 35,000-image, 8.4 GB subset of GenImage [2]: 17,500 real ImageNet photographs and 2,500 images from each of **seven** generators — ADM, BigGAN, GLIDE, Midjourney, SD 1.5, VQDM and Wukong. Every row carries its generator label. Inspecting the files directly, we confirmed that each generator emits exactly one resolution while real images vary, and that all images are JPEG except ADM, which is PNG. GenImage's format bias is therefore largely absent here, which is what makes this subset useful for our purpose: **size is the only substantial surviving confound**.

	BigGAN	ADM · GLIDE · VQDM	SD 1.5 · Wukong	Midjourney	Real (ImageNet)
Native resolution	128^2	256^2	512^2	1024^2	variable (e.g. 500×375)

Table 1. Native resolutions. Class and generator are both readable from size alone — this is the confound the project is about. Only the real class has variable aspect ratio, a fact that becomes decisive for the pad arm in §5.

Splits. The dataset ships a `train` split of 28,000 and a `validation` split of 7,000. We use the shipped `validation` split as our **test** set and carve our own validation set from 10% of `train`, giving per run 3,600 training / 400 validation / 4,000 test images. Model selection uses the internal validation set only, so no reported number ever influenced training. Each matrix cell pairs the target generator's 500 test fakes with the **same fixed 500 real images**, so every cell is exactly balanced at n = 1,000 and accuracy is directly interpretable; the binomial standard error is ±1.6 pp at p = 0.5 and ±1.0 pp at p = 0.9.

4 Models and hyperparameters

Architecture. A compact CNN trained **from scratch** in PyTorch, 1,173,473 parameters: four blocks of [Conv 3×3 → BatchNorm → ReLU] ×2 followed by 2×2 max-pooling, with widths 32/64/128/256 taking the 128×128 input down to 8×8, then global average pooling, dropout 0.3, and a single linear unit emitting one logit. Training from scratch is not a shortcut but a requirement: the real class *is* ImageNet, so any ImageNet-pretrained backbone would arrive already having seen the negative class. Two design choices follow from the nature of the signal. Nothing downsamples before the end of the first block, because the real-versus-generated cue is high-frequency and early pooling would destroy it; and the head is global average pooling rather than a flatten, because the evidence is a location-free texture statistic rather than an object in a particular place. The receptive field at the final block is 76×76, 59% of the image width.

Optimizer	AdamW	**Learning rate**	3×10^{-4}	**Weight decay**	1×10^{-4}
Scheduler	Cosine → 0	**Batch size**	128	**Epochs**	40
Loss	BCE-with-logits	**Precision**	AMP float16	**Augmentation**	h-flip p = 0.5 only
Normalization	(x/255 − 0.5)/0.5	**Selection**	best internal-val acc.	**Hardware**	Colab T4, torch 2.11.0

Table 2. Training configuration, identical across all 56 runs. Horizontal flip is the only augmentation by design: every other standard transform (random resized crop, JPEG jitter, blur) resamples or re-encodes the image and would inject the very artifact under study.

The four size-equalization strategies. Each takes a native-resolution image to 128×128. `center_crop` takes the central 128×128 window; `random_crop` takes a window at a random offset drawn once per image from an index-seeded RNG; `rescale` resizes the whole image bilinearly; `pad` scales the long side to 128 preserving aspect ratio and zero-pads to square. The first two

preserve native pixel statistics and discard content; the last two resample, and `pad` additionally introduces a border. Two structural properties of this design carry weight later. First, **all seven generators emit square images**, so `pad` and `rescale` are identical on every generated image and differ only on the real class — making that pair a clean intervention on real photographs alone. Second, BigGAN is natively 128², so all four strategies are the *identity* on its entire row, and any variation there comes purely from the real class.

Baselines and protocol. Two zero-parameter rules fix how much of the task is explained by size alone: the *square rule* predicts synthetic iff the image is square, and the *size-lookup rule* predicts synthetic iff the test resolution appears among the sizes the training generator emits. The full design is 4 strategies × 7 source generators × 2 seeds = **56 training runs**, each evaluated on all seven targets, yielding four 7×7 transfer matrices. Total compute was approximately 2 GPU-hours (127 s per run) on a free-tier T4.

5 Results

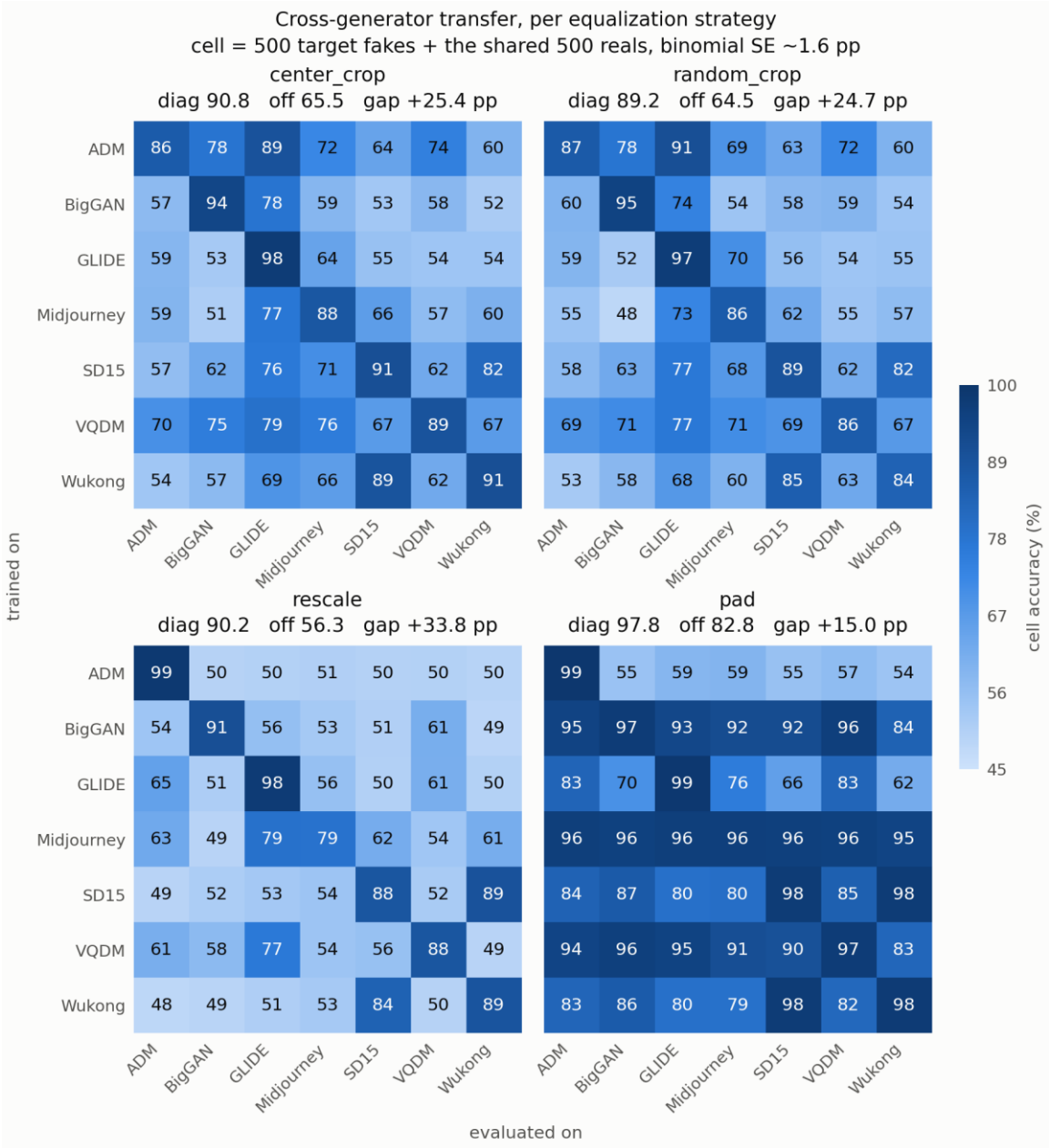

Figure 1. Cross-generator transfer accuracy under each size-equalization strategy (rows = training generator, columns = evaluation generator; mean of two seeds; every cell n = 1,000, balanced). The four panels share one colour scale. Off-diagonal structure — not the diagonal — is what differs between strategies.

Method	Diagonal (in-domain)	Off-diagonal (transfer)	Gap
`center_crop`	0.908	0.655	0.254
`random_crop`	0.892	0.645	0.247
`rescale`	0.902	**0.563**	0.338
`pad`	0.978	0.828	0.150
square rule (0 parameters)	0.977	**0.977**	0.000
size-lookup rule (0 parameters)	1.000	0.619	0.381

Table 3. In-domain versus cross-generator accuracy. The square rule, which never looks at a pixel, transfers better than every trained detector except pad — and pad's advantage is an artifact, not a success (see below).

The confound is real and it is resolution. The square rule scores **0.977** uniformly across all 49 cells (TPR 1.000, TNR 0.954): every generated image in this dataset is square and 95.4% of real photographs are not. The size-lookup rule is sharper still and exposes the mechanism exactly — it scores **1.000 whenever the source and target generators share a native resolution and exactly 0.500 whenever they do not**. Nothing about the generative process is being measured by these rules; only image dimensions are. This is the shortcut every trained detector below is competing against.

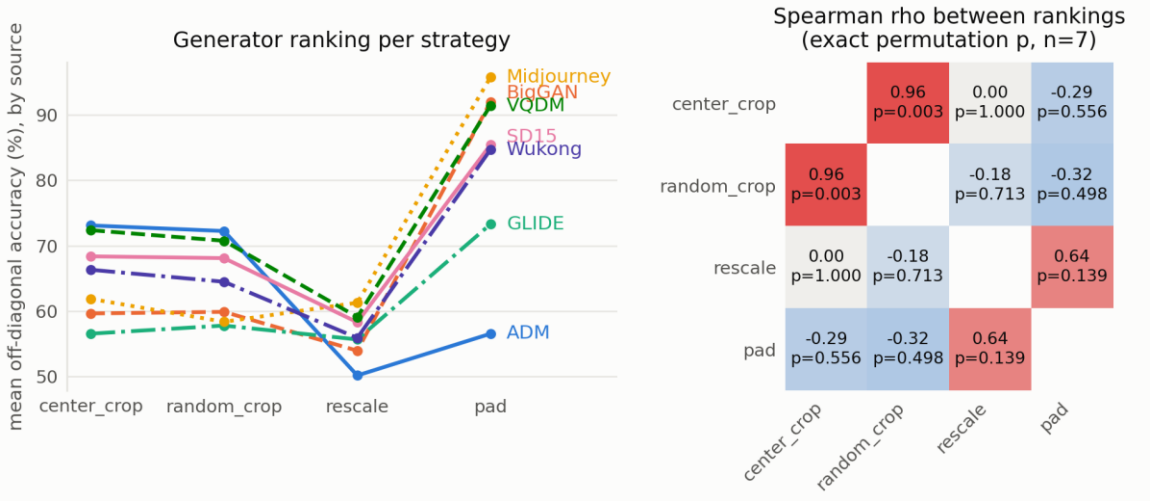

Figure 2. Left: mean off-diagonal accuracy of each generator as a training source, under each strategy; vertical order is the ranking. Right: Spearman ρ between the four rankings with exact permutation p-values. The two non-resampling strategies almost coincide (ρ = +0.964); rescale reorders them, and ADM falls from first to last.

Strategy pair	ρ (exact p)	Strategy pair	ρ (exact p)
`center_crop` ↔ `random_crop`	**+0.964 (0.003)**	`center_crop` ↔ `pad`	-0.286 (0.556)
`center_crop` ↔ `rescale`	+0.000 (1.000)	`random_crop` ↔ `pad`	-0.321 (0.498)
`random_crop` ↔ `rescale`	-0.179 (0.713)	`rescale` ↔ `pad`	+0.643 (0.139)

Table 4. Rank agreement between strategies. p-values are exact permutation p-values (n = 7), not a t-approximation.

The ranking is not stable under the correction. This is the main result. The two strategies that do *not* resample — `center_crop` and `random_crop` — agree almost perfectly on which generators yield transferable detectors (ρ = +0.964, p = 0.003). That agreement is the anchor: it shows the ranking is a stable, reproducible property of the data rather than seed noise. Against that anchor, `rescale` produces a ranking that retains none of the same structure (ρ = 0.000), and the movement is not a mild reshuffle — **ADM is the best source generator under both cropping strategies and the worst under rescaling**, a first-to-last inversion across seven positions. A paper reporting cross-generator difficulty after rescaling and a paper reporting it after cropping would disagree about which generator is hardest, using the same data, the same architecture and the same protocol.

We are deliberately careful about what the ρ = 0.000 supports. With n = 7 the exact permutation p-value is 1.000, which is *absence of evidence for an association, not evidence of independence*; seven points cannot establish a null. The claim rests instead on the positive finding — the +0.964 agreement between the two non-resampling arms, which is significant at n = 7 — together with the size of the movement. Per-cell seed disagreement averages 0.012–0.024 accuracy and never exceeds 0.076, so a seven-position inversion is far outside seed noise.

pad does not fix the confound; it re-encodes it. Table 3 shows `pad` with by far the best transfer (**0.828**), and reporting that as the best correction would be the central mistake this project exists to catch. Because every generator is square and only real photographs have variable aspect ratio, padding gives a black border to real images and to nothing else: **the border is a perfect proxy for the real class**. A detector that reads only the border would ceiling at (1.000 + 0.954)/2 = 0.977 — and pad's measured diagonal is 0.978. Two further observations agree. Transfer between generators sharing a native resolution exceeds transfer between generators that do not by +0.113, +0.113 and +0.135 for center_crop, random_crop and rescale, but by only **+0.007** for pad — pad's transfer is indifferent to generator similarity, as a border-reader would be. And pad converges fastest, at a mean best epoch of 26.6 versus 32.6–33.6 for the others, consistent with an easier surrogate cue. Higher accuracy here marks a worse experiment, not a better detector.

6 Analysis

Is the low transfer just an undertrained small CNN? No. Scoring each saved checkpoint on its own training set separates the three regimes and rules out a capacity explanation.

	Train	In-domain val	Cross-generator
`center_crop`	0.967	0.910	**0.655**
`rescale`	0.982	0.905	**0.563**

Table 5. Three-level decomposition. rescale fits its training set better and matches center_crop on held-out images of the same generator, then collapses only when the generator changes.

At 96.7–98.2% training accuracy the network is close to memorizing its 3,600 images, so the modest in-domain accuracy is a variance limit, not a bias limit; widening the model would push training accuracy toward 100% without moving validation. This forecloses the obvious alternative explanation — that the low diagonal simply reflects an undertrained network. More importantly, the decomposition sharpens the headline: **resampling does no measurable in-distribution damage**. rescale fits training data *better* than center_crop (0.982 vs 0.967) and generalizes equally well to held-out images of the same generator

(0.905 vs 0.910). The harm is invisible until the generator changes, where the two arms separate by 0.092. A practitioner validating in-distribution would see no reason for concern.

[TO VERIFY BEFORE SUBMISSION — Ido] The six numbers in Table 5 are recorded in PROJECT_STATE.md but have no committed artifact behind them: notebooks/02_train_val_gap.ipynb has no stored outputs and nothing was written under results/. Please confirm the values and commit the script output, or re-run src/trainval.py, before this is handed in.

An open observation. ADM behaves inconsistently across arms in a way we can measure but not yet explain. Under center_crop it is the hardest generator in the study (in-domain 0.863); under rescale it is nearly trivial (0.986) while its transfer collapses to chance (0.501, the lowest row mean anywhere). ADM is also the only PNG generator in a dataset that is otherwise JPEG — but that asymmetry predicts ADM should be easiest under center_crop, where native pixels and real-image JPEG blocking both survive intact, and we observe the opposite. Since ADM's row feeds directly into the ranking, we report this as an unresolved observation rather than offer a mechanism we have not tested.

Cue-level evidence. [PLACEHOLDER — to be completed] Input-gradient attribution (saliency = max-channel $|\partial f(x)/\partial x|$, averaged over 200 images per generator per strategy) and radially averaged power spectra were proposed to test directly whether the cues shift between strategies — in particular whether the pad model's saliency concentrates in the outer 16-pixel ring, which would confirm the border reading argued for in §5 from behaviour alone. These analyses have not yet been run.

[PLACEHOLDER — Noa] Replace the paragraph above once attribution and spectra are run, or, if they are dropped, state the omission explicitly here since both were promised in the submitted proposal. Attribution needs the center_crop and rescale checkpoints, which are on Ido's Drive.

7 Discussion: insights, limitations and future directions

Insights. Three findings survive our controls. (i) On this benchmark a zero-parameter size rule transfers better than three of four trained detectors, so cross-generator numbers reported without an explicit size correction are largely uninterpretable. (ii) The correction is not a neutral preprocessing detail: strategies that preserve native pixels agree on generator difficulty ($\rho = +0.964$) while resampling produces an unrelated ordering, so the ranking a paper publishes is partly a function of a choice that is rarely reported. (iii) A correction can raise accuracy while making the experiment worse — pad posts the best transfer in our study by substituting a border cue for the size cue. The practical recommendation that follows is that cross-generator work should state its size-equalization strategy as a first-class methodological choice, and preferably report more than one.

Limitations. We test one architecture at one input resolution; a larger or pretrained backbone might rank generators differently, though pretraining is unavailable here because the real class is ImageNet. Each cell carries two seeds, enough to place a seven-position inversion outside seed noise (mean per-cell seed disagreement 0.012–0.024) but not enough for tight confidence intervals on individual cells (±1.6 pp binomial SE alone). The ranking correlations rest on $n = 7$ generators, which is the width of the benchmark, not a design choice; $\rho = 0.000$ should be read as absence of evidence rather than proof of independence. We use a 35,000-image subset rather than full GenImage, and the cue-level analyses in §6 are outstanding, so the border-reading account of pad rests on behavioural evidence only. Finally, the ADM anomaly is unexplained.

Future directions. The immediate next step is the attribution and spectral analysis of §6, which would convert the pad argument from inference to direct evidence. Beyond that: resolving ADM by separating format from resolution (re-encoding ADM as JPEG and re-running its row) would isolate the one remaining format confound; adding a resolution-preserving strategy that crops at native scale and evaluates at several scales would separate "resampling destroys the fingerprint" from "cropping discards context"; and repeating the four-arm comparison on full GenImage with a ResNet50 would test whether the reordering we observe persists at the scale and capacity used in the literature.

8 References

[1] S.-Y. Wang, O. Wang, R. Zhang, A. Owens and A. A. Efros. CNN-generated images are surprisingly easy to spot… for now. CVPR 2020. arXiv:1912.11035.

[2] M. Zhu, H. Chen, Q. Yan, X. Huang, G. Lin, W. Li, Z. Tu, H. Hu, J. Hu and Y. Wang. GenImage: A Million-Scale Benchmark for Detecting AI-Generated Image. NeurIPS 2023, Datasets and Benchmarks Track. arXiv:2306.08571.

[3] P. Grommelt, L. Weiss, F.-J. Pfreundt and J. Keuper. Fake or JPEG? Revealing Common Biases in Generated Image Detection Datasets. 2024. arXiv:2403.17608.

[4] Tiny-GenImage dataset. Hugging Face, TheKernel01/Tiny-GenImage. Licence CC BY-NC-SA 4.0.

9 Contribution of each team member

Ido Josephsberg	**Noa Keter**
Data pipeline and cache build (src/data.py); training loop (src/train.py); training runs for the center_crop and rescale arms (28 runs); train/validation diagnostic (src/trainval.py); analysis and figure code (src/analyze.py); repository README and reproducibility instructions.	Model architecture (src/model.py); zero-parameter baselines (src/baseline.py); training runs for the pad and random_crop arms (28 runs); rotation-sensitivity ablation (src/rotation.py); cue-level analysis (attribution and spectra); report sections on related work, data, analysis and discussion.
Report: motivation, models and hyperparameters, results.	Report: assembly, references, contribution table.

[TO CONFIRM — both] Check this table against what each of you actually did before submitting; the course states that an unbalanced or inaccurate contribution split can affect individual grades.

Code and reproducibility. All code, the exact run commands, the split definitions and every metrics.json produced by the 56 runs are in the public repository at `https://github.com/noa-keter/deep-learning`. Seeds are fixed and the cache build is deterministic, so each reported cell reproduces from a clean clone.